Web 2 – Séance 04

Hachage et Sécurité

❌ NE PAS : Stocker les mots de passe en clair

```
// DANGER ! Cela expose les données sensibles
interface User {
  id: number;
  email: string;
  password: string; // ❌ JAMAIS en clair !
}

const users = [
  { id: 1, email: "john@gmail.com", password: "secret123" },
  { id: 2, email: "jane@gmail.com", password: "password456" },
];
```

Si la BDD est piratée, tous les mots de passe sont compromis.
Et si l'utilisateur réutilise ce mot de passe ailleurs… catastrophe !

À la place, on stocke un **hash** du mot de passe, pas le mot de passe lui-même.

```
interface User {
  id: number;
  email: string;
  passwordHash: string; // ✅ Stocker le hash du mot de passe
}
```

Hachage vs Chiffrement

- **Chiffrement (symétrique)** : Un texte **chiffré** avec une clé secrète peut être **déchiffré** avec la même clé.
- **Hachage** : Un texte **haché** avec un algorithme ne peut pas être **déchiffré**. C'est unidirectionnel.
  - On peut cependant comparer un texte avec son hash pour vérifier si c'est le même.

Propriétés d’un bon algorithme de hachage

hash = H(entrée)

- **Taille fixe** (ex: 256 bits)
- **Unicité** : unique pour chaque entrée (collision rare)
- **Irréversibilité** : on ne peut pas retrouver l’entrée à partir du hash
- **Déterminisme** : même entrée → même hash
- **Chaotique** : petit changement dans l’entrée → hash complètement différent
- **Coûteux en calcul** : volontairement lent, pour ralentir les attaques par force brute

Hachage avec Salt

- **Sans Salt** : Opération déterministe, même mot de passe → même hash
  - Si deux utilisateurs ont le même mot de passe, ils auront le même hash
  - Attaques par dictionnaire : tester tous les mots de passe courants et comparer si le hash correspond
  - Rainbow tables : table pré-calculée de mots de passe et leurs hash, pour retrouver rapidement le mdp
- **Avec Salt** : on ajoute une valeur aléatoire au mot de passe avant de le hacher
  - Stockée dans la BDD avec le hash, pour pouvoir vérifier le mot de passe plus tard
  - Modifie le hash même si deux utilisateurs ont le même mot de passe
  - Protège contre les attaques par dictionnaire et rainbow tables

Librairie `bcrypt`

- Installation :

```
npm install bcrypt
npm install --save-dev @types/bcrypt
```

- Importation :

```
import bcrypt from "bcrypt";
```

BCrypt fait tout automatiquement :

- Génère un salt aléatoire
- Ajoute le salt au mot de passe
- Hache plusieurs fois :
  - Coût = nombre de tours de hachage
  - Augmente le temps de calcul pour ralentir les attaques par force brute
  - Chaque +1 double le temps
- Retourne salt + hash en un seul string

Utilisation de bcrypt

```
import bcrypt from "bcrypt";

// Hacher un mot de passe
const password = "MySecretPassword123!";
const saltRounds = 10; // coût de hachage
bcrypt.hash(password, saltRounds); // retourne une Promise<string>

// Vérifier un mot de passe
const tentativePassword = "MySecretPassword123!";
const storedHash = "$2b$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcg7b3XeKeU6xBJxvxaXUtSQm1S";
bcrypt.compare(tentativePassword, storedHash); // retourne une Promise<boolean>
```

Pourquoi bcrypt doit être asynchrone

- Hacher un mot de passe est **volontairement lent** (~100 ms), pour freiner les attaques par force brute
- JavaScript est **single-threaded** : un seul thread exécute notre code
- Version **synchrone** : ce thread fait le calcul lui-même → le serveur ne peut rien faire d'autre en attendant
- Version **asynchrone** : bcrypt confie le calcul à **un autre thread** (code C++) → le thread principal reste libre
- `await` **met la fonction en pause** ; elle **reprend** automatiquement quand le hash est prêt

```
// ❌ Version synchrone, pendant 100 ms, plus personne ne peut se connecter
const hash = bcrypt.hashSync(password, 10);

// ✅ Version asynchrone, le serveur continue de répondre pendant le calcul
const hash = await bcrypt.hash(password, 10);
```

Les fonctions asynchrones sont également utilisées pour les opérations lentes qui nécessitent d'attendre une réponse d'un serveur ou d'une base de données. Le serveur peut continuer à répondre à d'autres requêtes pendant ce temps, et la fonction reprend automatiquement quand la réponse est prête.

Fonction asynchrone : Promises

Une fonction asynchrone retourne une **Promise**.

- Une Promise est un objet qui représente une valeur qui sera disponible dans le futur
- Les Promises peuvent être dans 3 états :
  1. **Pending** : en attente, pas encore résolue
  2. **Fulfilled** : résolue avec succès, valeur disponible
  3. **Rejected** : rejetée avec une erreur, valeur non disponible
- Le résultat d'une Promise est récupéré via des **callbacks**
  - **then()** : pour gérer le succès
  - **catch()** : pour gérer l'erreur

Fonction asynchrone : Promises (exemple)

```
// Bcrypt.hash() retourne une Promise
const hashPromise: Promise<string> = bcrypt.hash("MySecretPassword123!", 10);

// Ici, la Promise est en état "pending" (en attente)

hashPromise.then((hash) => {
  console.log("Hash généré :", hash); // Ici, la Promise est "fulfilled" (résolue)
}).catch((error) => {
  console.error("Erreur lors du hachage :", error); // Ici, la Promise est "rejected" (rejetée)
});
```

Une fonction qui fait appel à une Promise ne peut pas retourner directement la valeur.
À la place, elle pourrait retourner une Promise elle-même, ou accepter une fonction callback.

```
function hashPassword(plainPassword: string, callback: (hash: string | null) => void) {
  bcrypt.hash(plainPassword, 10).then((hash) => {
    callback(hash);
  }).catch((error) => {
    callback(null);
  });
}
```

Fonction asynchrone : async/await

Autre syntaxe pour gérer les Promises qui est plus lisible et évite les callbacks imbriqués : `async/await`

- Une fonction déclarée avec `async` :
  - Retourne automatiquement une Promise
  - Peut utiliser `await` dans son corps
- Le mot-clé `await` :
  - Peut être utilisé uniquement dans une fonction `async`
  - Permet d'attendre la résolution d'une fonction asynchrone (Promise) avant de continuer l'exécution du code

```
async function hashPassword(plainPassword: string): Promise<string> {
    try {
        const hash: string = await bcrypt.hash(plainPassword, 10);
        return hash;
    } catch (error) {
        throw new Error("Erreur lors du hachage");
    }
}
```

Inscription avec bcrypt

```
class UsersService {
  static async create(newUser: NewUser): Promise<User | undefined> {
    if (this.getByEmail(newUser.email)) return undefined;

    // Hacher le mot de passe avant de le stocker
    const passwordHash = await bcrypt.hash(newUser.password, 10);
    const user: User = {
      id: UsersService.getNextId(users),
      email: newUser.email,
      password: passwordHash, // Stocker le hash du mot de passe
      firstName: newUser.firstName,
      lastName: newUser.lastName,
      role: ERole.USER,
      favorites: [],
      createdAt: new Date(),
      updatedAt: new Date(),
    };

    const users = this.readUsersDB();
    users.push(user);
    if (!this.writeUsersDB(users)) return undefined;
    return user;
  }
}
```

Controller l’inscription avec bcrypt

```
authController.post("/register", async (req: Request, res: Response) ⇒ {
  const body = req.body;
  if (!isNewUserDTO(body)) return res.sendStatus(400);
  const newUser: NewUser = body;

  const user = await UsersService.create(newUser);
  if (!user) return res.sendStatus(409); // Conflit : email déjà utilisé

  // Générer un token JWT pour l'utilisateur
  const token = AuthService.login(user.email, user.password);
  if (!token) return res.sendStatus(500); // should not happen

  return res.status(201).json({ token });
});
```

Connexion avec bcrypt

```
class AuthService {
  static async login(email: string, password: string): Promise<string | undefined> {
    // Récupérer l'utilisateur par email
    const user = UsersService.getByEmail(email);
    if (!user) return undefined; // Utilisateur non trouvé

    // Vérifier le mot de passe avec le hash stocké
    const isMatch = await bcrypt.compare(password, user.passwordHash);
    if (!isMatch) return undefined; // Mot de passe incorrect

    // Générer un token JWT pour l'utilisateur
    return generateToken({
      id: user.id,
      email: user.email,
      role: user.role,
    });
  }
}
```

Route de connexion avec bcrypt

```
authController.post("/login", async (req: Request, res: Response) => {
  const body: unknown = req.body;
  if (!isCredentialsDTO(body)) return res.sendStatus(400);

  const { email, password } = body;
  const token = await AuthService.login(email, password);
  if (!token) return res.sendStatus(401); // Non autorisé : email ou mot de passe incorrect

  res.json({ token });
});
```

Flux Complet : Register

```
### 1. Register (créer un compte)
# @name register
POST /auth/register
Content-Type: application/json

{
  "email": "john@gmail.com",
  "password": "MySecurePassword123!"
}

###

# Serveur hache le mot de passe avec bcrypt
# Stocke : user{ email, passwordHash: "$2b$10$..." }
# Retourne : { token: "eyJh..." }

# 2. Client stocke le token
```

Flux Complet : Login

```
### 3. Login (se connecter)
# @name login
POST /auth/login
Content-Type: application/json

{
  "email": "john@gmail.com",
  "password": "MySecurePassword123!"
}

###

# Serveur compare password avec bcrypt.compare()
# Génère nouveau token
# Retourne : { token: "eyJh..." }

# 4. Client stocke le token
```

Flux Complet : Accès protégé

```
### 5. Accès route protégée
GET /recipes
Authorization: {{login.response.body.token}}

# Middleware vérifie le token
# Retourne : [ ... recipes]

### 6. Suppression protégée
DELETE /recipes/5
Authorization: {{login.response.body.token}}

# Middleware vérifie le token
# Route vérifie autorisation (est-ce l'auteur ?)
# Supprime
```

Récapitulatif Séance 04

- **Hachage vs Chiffrement** – Unidirectionnel vs bidirectionnel
- **Propriétés d'un bon algorithme de hachage** – Taille fixe, unicité, irréversibilité, déterminisme, chaotique, calculatoire
- **Salt** – Valeur aléatoire pour protéger contre les attaques par dictionnaire et rainbow tables
- **bcrypt** – Library de hachage avec salt intégré
- **Fonctions asynchrones** – Promises et async/await pour ne pas bloquer le serveur
- **Route d'inscription** – Hacher le mot de passe avec bcrypt avant de stocker
- **Route de connexion** – Comparer le mot de passe avec bcrypt.compare() pour vérifier l'identité

Prochaine séance : Séance 05 – Query Selector (frontend "old school", avant de passer à React).

Exercice filé S04

1. Ajoutez le hachage des mots de passe avec bcrypt dans votre backend
2. Testez entièrement votre backend
3. Vérifiez que les mots de passe ne sont plus stockés en clair dans la BDD